

BSc (Hons) in **Computer Science****SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing**Practical 01**

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation**Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure
Environment
Actuator
Sensor

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
width
height
walls
agent_pos
food_position
opponents
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent - Because the environment contains multiple agents: the main agent and opponent agents. The opponents can independently perform actions and affect the main agent's outcome through collisions.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The complete history of all percepts received by an agent from the beginning of its operation until the current time.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable, because the `get_percept()` method only provides limited information to the agent, such as `agent_pos`, `opponent_positions`, `smells_food`, `hit_wall`, `collision`, `score`, and `remaining_food`. It does not provide the complete physical environment state, such as the full food locations or the complete wall layout. Therefore, the agent cannot observe the entire environment and must act using incomplete information.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important because an AI agent is evaluated based on its effect on the environment and its ability to achieve its goals, not on how much internal computation it performs. In this code, hitting a wall changes the external environment interaction, so reducing the score reflects a real consequence of the agent's action. Measuring internal processing effort would not show whether the agent is behaving successfully in the environment.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

It makes the environment more Partially Observable.

Adding `self.toxic_traps` creates new hidden information in the environment. Since the agent's sensors (`get_percept()`) do not include the trap locations, the agent cannot observe the complete environment state. Therefore, the agent must make decisions with incomplete information, making the environment Partially Observable.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.
10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding the `smells_toxin` sensor helps the agent act more rationally because it gives the agent additional information about the environment, allowing it to make better decisions that maximize its expected utility. Without this sensor, the agent may unknowingly move into toxic traps and lose points. With the sensor, the agent can detect danger and choose safer actions, improving its chances of achieving a higher score and avoiding penalties.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World exploit demonstrated that if a performance metric is poorly designed, an agent may find a way to maximize its score without achieving the real objective. For example, a vacuum agent could repeatedly clean the same area to gain reward instead of actually cleaning the environment. This shows why penalties, such as the toxic trap score reduction, are needed to ensure the metric encourages the correct behaviour.

Finalize and Lock Lab 01 Analytical Evaluation

Incomplete: 10/11 questions answered. Please provide detailed responses for all questions.

